Chapitre 3 : Structures et Unions

Programmation C Avancée - L3

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop de Dakar

1 Structures

1.1 Déclaration et Définition

Une structure est un type composé regroupant des variables de types différents sous un même nom. La déclaration définit le layout mémoire.

```c
// Déclaration de structure
struct Point {
    int x;
    int y;
};

// Définition de variable
struct Point p1;

// Déclaration + définition simultanée
struct Point p2 = {10, 20};
```

Syntaxe de déclaration

```c
struct nom_structure {
    type1 membre1;
    type2 membre2;
    // ...
};
```

La structure crée un nouveau type. Chaque membre a un offset fixe par rapport au début de la structure.

1.2 Initialisation

```c
// Initialisation positionnelle
struct Point p1 = {10, 20};

// Initialisation désignée (C99)
struct Point p2 = {.y = 20, .x = 10};

// Initialisation partielle (membres restants à 0)
struct Point p3 = {.x = 5};  // y = 0

// Initialisation composée (compound literal, C99)
struct Point p4 = (struct Point){100, 200};
```

Initialisation désignée (C99)

Les designated initializers permettent d'initialiser les membres dans n'importe quel ordre et de n'initialiser qu'un sous-ensemble. Les membres non spécifiés sont initialisés à zéro.

1.3 Accès aux Membres

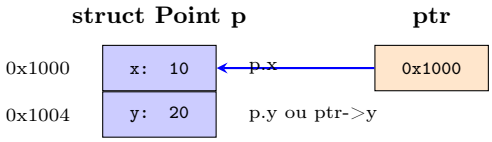
```c
struct Point p = {10, 20};
struct Point *ptr = &p;

// Accès direct (opérateur .)
int a = p.x;       // a = 10

// Accès via pointeur (opérateur ->)
int b = ptr->y;    // b = 20

```

```c
// Équivalence: ptr->membre == (*ptr).membre
int c = (*ptr).x;  // c = 10
```



1.4 typedef

```c
// Sans typedef
struct Point {
    int x, y;
};
struct Point p1;  // mot-clé struct requis

// Avec typedef
typedef struct {
    int x, y;
} Point;
Point p2;  // plus besoin de struct

// typedef avec nom de structure (pour auto-référence)
typedef struct Node {
    int data;
    struct Node *next;  // auto-référence
} Node;
```

2 Alignement et Padding

2.1 Règles d'Alignement

Les processeurs modernes imposent des contraintes d'alignement pour des raisons de performance. Un type de taille n octets doit généralement être aligné sur une adresse multiple de n.

Contraintes d'alignement (x86-64)

Type	Taille (octets)	Alignement
char	1	1
short	2	2
int	4	4
long	8	8
float	4	4
double	8	8
pointeur	8	8

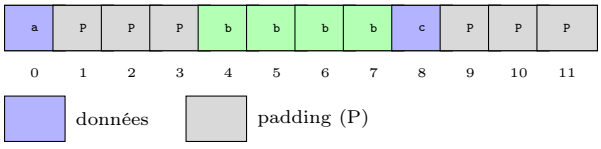
L'alignement d'une structure est celui de son membre le plus contraignant.

2.2 Calcul du Padding

Le compilateur insère des octets de padding pour respecter l'alignement de chaque membre.

```c
struct S1 {
    char a;     // offset 0, size 1
    int b;      // offset 4 (padding: 3), size 4
    char c;     // offset 8, size 1
};              // taille totale: 12 (padding final: 3)
```

struct S1 (sizeof = 12)



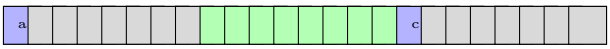
2.3 Optimisation par Réorganisation

Réorganiser les membres par taille décroissante minimise le padding.

```
// Structure non optimisée: 24 octets
struct Bad {
    char a;      // 1 + 7 padding
    double b;    // 8
    char c;      // 1 + 7 padding
};

// Structure optimisée: 16 octets
struct Good {
    double b;    // 8
    char a;      // 1
    char c;      // 1 + 6 padding
};
```

struct Bad (24 octets)



struct Good (16 octets)



2.4 Représentation Hexadécimale

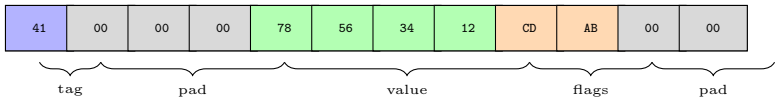
Analyse forensique d'une structure

```
struct Data {
    char tag;       // offset 0
    int value;      // offset 4
    short flags;    // offset 8
};
struct Data d = {'A', 0x12345678, 0xABCD};
```

Dump mémoire (little-endian, adresse 0x1000):

```
0x1000: 41 00 00 00 78 56 34 12 CD AB 00 00
        ^  ^--padding  ^--value     ^--flags ^--padding
        tag
```

struct Data en mémoire



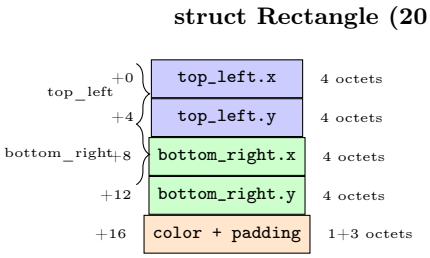
3 Structures Imbriquées

```
struct Point {
    int x, y;
};

struct Rectangle {
    struct Point top_left;
    struct Point bottom_right;
    char color;
};

struct Rectangle r = {{0, 0}, {100, 50}, 'R'};

// Accès
int x1 = r.top_left.x;
int y2 = r.bottom_right.y;
```

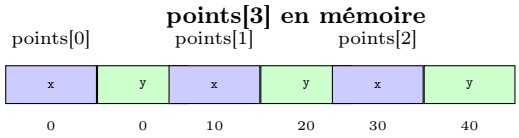


3.1 Tableaux de Structures

```c
struct Point points[3] = {{0,0}, {10,20}, {30,40}};

// Accès
int x1 = points[1].x;      // 10
int y2 = points[2].y;      // 40

// Via pointeur
struct Point *p = points;
int x0 = p->x;             // 0
int x1 = (p+1)->x;         // 10
```



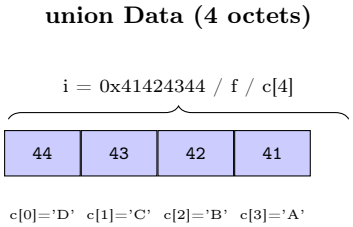
4 Unions

4.1 Définition et Caractéristiques

Une union est un type composé où tous les membres partagent la même zone mémoire. La taille de l'union est celle de son plus grand membre.

```c
union Data {
    int i;
    float f;
    char c[4];
};

union Data d;
d.i = 0x41424344;
printf("%c%c%c%c\n", d.c[0], d.c[1], d.c[2], d.c[3]);
// Affiche: DCBA (little-endian)
```



Type Punning

Accéder à un membre différent de celui qui a été écrit en dernier constitue du type punning. Le comportement est défini en C (contrairement à C++) mais peut produire des résultats dépendants de l'architecture (endianness).

4.2 Tagged Union (Union Discriminée)

Pattern pour utiliser les unions de manière sûre: associer un tag indiquant le type actif.

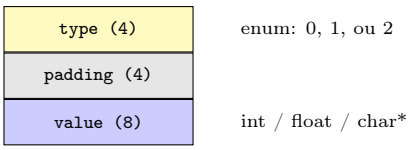
```c
typedef enum { INT_TYPE, FLOAT_TYPE, STRING_TYPE } DataType;

typedef struct {
    DataType type;  // discriminant
    union {
        int i;
        float f;
        char *s;
```

```c
    } value;
} Variant;

void print_variant(Variant *v) {
    switch (v->type) {
        case INT_TYPE:    printf("%d\n", v->value.i); break;
        case FLOAT_TYPE:  printf("%f\n", v->value.f); break;
        case STRING_TYPE: printf("%s\n", v->value.s); break;
    }
}
```

Variant (16 octets)



4.3 Cas d'Usage des Unions

Applications typiques

- **Type punning**: conversion entre représentations (int ↔ float)
- **Économie mémoire**: stocker des données alternatives
- **Protocoles réseau**: interpréter des paquets selon le type
- **Registres hardware**: accéder à des champs de bits
- **Variants**: implémenter des types polymorphes

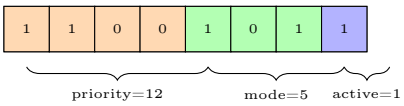
5 Champs de Bits (Bit Fields)

Les bit fields permettent de définir des membres de structure occupant un nombre spécifique de bits.

```c
struct Flags {
    unsigned int active : 1;    // 1 bit
    unsigned int mode : 3;      // 3 bits (0-7)
    unsigned int priority : 4;  // 4 bits (0-15)
};

struct Flags f = {1, 5, 12};
// Layout possible: [priority:4][mode:3][active:1] = 0xC5
```

Représentation en bits



Portabilité des Bit Fields

L'ordre des bits, l'alignement, et le packing des bit fields sont définis par l'implémentation. Le code utilisant des bit fields pour des protocoles réseau ou des formats de fichiers n'est pas portable. Préférer les masques et décalages explicites pour le code portable.

6 Structures et Allocation Dynamique

6.1 Allocation de Structures

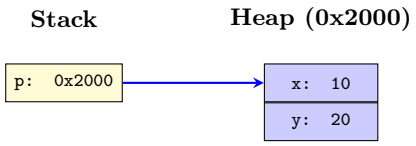
```c
#include <stdlib.h>

struct Point {
    int x, y;
};

// Allocation d'une structure
```

```c
struct Point *p = malloc(sizeof(struct Point));
if (p == NULL) { /* erreur */ }
p->x = 10;
p->y = 20;
free(p);

// Allocation d'un tableau de structures
struct Point *points = malloc(5 * sizeof(struct Point));
for (int i = 0; i < 5; i++) {
    points[i].x = i;
    points[i].y = i * 2;
}
free(points);
```



6.2 Structures avec Membres Dynamiques

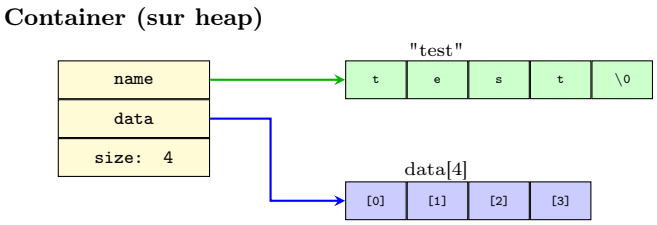
```c
typedef struct {
    char *name;
    int *data;
    size_t size;
} Container;

Container *create_container(const char *name, size_t n) {
    Container *c = malloc(sizeof(Container));
    if (!c) return NULL;

    c->name = strdup(name);  // allocation pour name
    c->data = malloc(n * sizeof(int));
    c->size = n;

    if (!c->name || !c->data) {
        free(c->name);
        free(c->data);
        free(c);
        return NULL;
    }
    return c;
}

void destroy_container(Container *c) {
    if (c) {
        free(c->name);  // libérer les membres d'abord
        free(c->data);
        free(c);        // puis la structure
    }
}
```



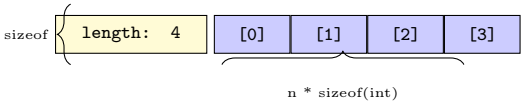
6.3 Flexible Array Member (C99)

```c
// Membre tableau flexible (doit être le dernier)
typedef struct {
    size_t length;
    int data[];  // flexible array member
} FlexArray;

// Allocation
FlexArray *create(size_t n) {
    FlexArray *arr = malloc(sizeof(FlexArray) + n * sizeof(int));
    arr->length = n;
    return arr;
}

FlexArray *a = create(10);
a->data[0] = 42;  // accès direct
```

FlexArray avec n=4



7 Structures Opaques et Encapsulation

7.1 Forward Declaration

La déclaration anticipée permet de déclarer un type sans révéler sa structure.

```c
// Dans header.h
typedef struct Stack Stack;  // forward declaration

Stack *stack_create(void);
void stack_push(Stack *s, int value);
int stack_pop(Stack *s);
void stack_destroy(Stack *s);

// Dans implementation.c
#include "header.h"

struct Stack {
    int *data;
    size_t capacity;
    size_t top;
};

Stack *stack_create(void) {
    Stack *s = malloc(sizeof(Stack));
    s->data = malloc(16 * sizeof(int));
    s->capacity = 16;
    s->top = 0;
    return s;
}
```

Avantages de l'encapsulation

- **Abstraction**: l'utilisateur manipule un pointeur opaque
- **Évolution**: la structure interne peut changer sans modifier l'API
- **Compilation**: modifier l'implémentation ne recompile pas les clients
- **Sécurité**: impossible d'accéder directement aux membres

8 Résumé des Tailles et Alignements

Concept	Taille	Alignement	Notes
`struct`	$\geq$ somme membres	max(membres)	+ padding
`union`	max(membres)	max(membres)	membres superposés
bit field	selon déclaration	selon type	non portable
flexible array	0 dans sizeof	type élément	C99

Outils d'inspection

```c
// offsetof: offset d'un membre (stddef.h)
#include <stddef.h>
size_t off = offsetof(struct S, membre);

// sizeof: taille totale
size_t sz = sizeof(struct S);

// GDB: examiner une structure
(gdb) ptype struct S
(gdb) p sizeof(struct S)
(gdb) x/12xb &variable
```